INFORMATIONSBLATT

IB01 | HTTPS und TLS – Grundlagen

LF10b · LS02 – HTTPS und TLS konfigurieren · IB01 – HTTPS und TLS Grundlagen | IT-3B · Schuljahr 2025/2026

> 📖 **LERNZIEL**
> Nach diesem Informationsblatt können Sie erklären, warum HTTPS notwendig ist, wie TLS eine verschlüsselte Verbindung aufbaut und welche Typen von Zertifikaten es gibt. Sie kennen den Unterschied zwischen HTTP und HTTPS und können ihn im beruflichen Kontext begründen.

1 | Warum HTTPS? Das Problem mit HTTP

HTTP (HyperText Transfer Protocol) überträgt alle Daten **unverschlüsselt** als Klartext. Das bedeutet: Jeder, der sich im selben Netzwerk befindet – oder den Netzwerkverkehr abhören kann – kann Benutzernamen, Passwörter, Formulardaten und sogar Sitzungs-Cookies mitlesen.

HTTPS (HTTP Secure) löst dieses Problem, indem HTTP-Kommunikation durch das **TLS-Protokoll** (Transport Layer Security) gesichert wird. HTTPS läuft standardmäßig auf **Port 443**, HTTP auf Port 80.

Eigenschaft	HTTP	HTTPS
Standardport	80	443
Verschlüsselung	Keine – Klartext	TLS – verschlüsselt
Abhörschutz	❌ Kein Schutz	✅ Inhalte nicht lesbar
Integrität	❌ Daten veränderbar	✅ Manipulation erkannt
Identitätsprüfung	❌ Keine	✅ Durch Zertifikat
Browser-Anzeige	⚠️ „Nicht sicher"	🔒 Schloss (CA-signiert) | ⚠️ Warnung (Self-Signed)

> ⚠️ **PRAXISRELEVANZ**
> Ohne HTTPS kann jeder im selben WLAN oder LAN-Segment die Verbindung mitlesen. Besonders bei internen Webportalen mit Login-Formularen oder sensiblen Daten ist HTTP ein erhebliches Sicherheitsrisiko – auch im Unternehmens-Intranet.

2 | TLS – Transport Layer Security

TLS ist das Protokoll, das die Verschlüsselung von HTTPS übernimmt. Es arbeitet zwischen der Transportschicht (TCP) und der Anwendungsschicht (HTTP) und bietet drei Schutzziele:

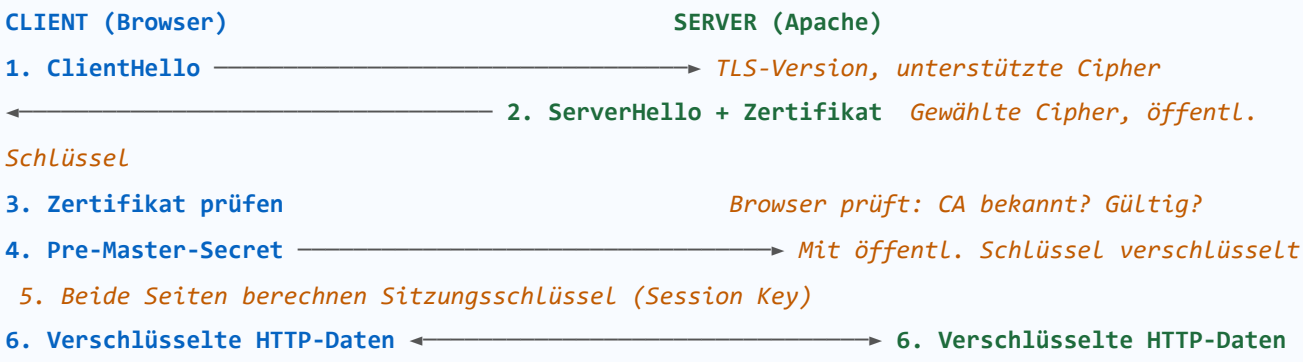
- **Vertraulichkeit:** Daten werden verschlüsselt – kein Mitlesen möglich.
- **Integrität:** Manipulationen an Datenpaketen werden erkannt (HMAC bzw. AEAD-Verfahren wie AES-GCM).

- **Authentizität:** Der Server weist seine Identität per Zertifikat nach.

> 💡 **MERKE – SSL vs. TLS**
> „SSL" (Secure Sockets Layer) ist der historische Vorläufer von TLS. SSL 2.0 und 3.0 sowie TLS 1.0 und 1.1 sind veraltet und unsicher. Aktuelle Server verwenden **TLS 1.2** oder **TLS 1.3**. Der Begriff „SSL-Zertifikat" ist dennoch weit verbreitet – gemeint ist immer ein TLS-Zertifikat.

2.1 | Der TLS-Handshake (vereinfacht)

Bevor Daten verschlüsselt übertragen werden, führen Client und Server einen **TLS-Handshake** durch. Dabei werden Verschlüsselungsalgorithmus und Sitzungsschlüssel ausgehandelt:



Ab Schritt 6 werden alle Daten mit dem gemeinsamen **Sitzungsschlüssel** symmetrisch verschlüsselt (z. B. AES-256). Nur Client und Server kennen diesen Schlüssel.

> ✅ **ZWISCHENÜBUNG – TLS-Handshake-Schritte beschriften**
> **Aufgabe:** Schreiben Sie zu jedem der folgenden Schritte des TLS-1.2-Handshakes in eigenen Worten auf, was dabei übertragen wird und welchen Zweck der Schritt hat. Nutzen Sie das Diagramm oben.

Schritt	Was wird übertragen?	Zweck
1 · ClientHello		
2 · ServerHello + Zertifikat		
3 · Zertifikat prüfen		
4 · Pre-Master-Secret		
5 · Session Key		

Vergleich: Ihr Lehrer bespricht die Lösungen im Plenum.

3 | TLS-Zertifikate

Ein TLS-Zertifikat ist eine digitale Datei, die die Identität eines Servers bestätigt und seinen öffentlichen Schlüssel enthält. Es funktioniert ähnlich wie ein Personalausweis: Eine vertrauenswürdige Stelle (CA) bestätigt, dass der Inhaber tatsächlich der ist, der er behauptet zu sein.

3.1 | Inhalt eines Zertifikats

Feld	Bedeutung	Beispiel
Common Name (CN)	Domain / Hostname des Servers	`netconsult.local`
Organization (O)	Name des Unternehmens	`NetConsult GmbH`
Country (C)	Länderkürzel (2 Buchstaben)	`DE`
Valid From / To	Gültigkeitszeitraum	2025-01-01 bis 2026-01-01
Public Key	Öffentlicher Schlüssel des Servers	RSA 2048 bit oder höher
Issuer	Ausstellende Zertifizierungsstelle (CA)	`Let's Encrypt` oder `Self-Signed`
Signature	Digitale Unterschrift der CA	SHA-256withRSA

3.2 | Zertifikatstypen im Vergleich

Typ	Beschreibung	Browser-Vertrauen	Kosten	Einsatz
Self-Signed	Selbst erstellt und signiert – keine CA	❌ Warnung im Browser	Kostenlos	Intern, Testumgebungen, Schullab
CA-signiert (DV)	Domain Validation – CA prüft Domain-Besitz	✅ Keine Warnung	Kostenlos (Let's Encrypt)	Öffentliche Webserver
CA-signiert (OV)	Organization Validation – CA prüft Firma	✅ Keine Warnung	50–300 €/Jahr	Unternehmenswebseiten
CA-signiert (EV)	Extended Validation – strenge Prüfung	✅ Keine Warnung	200–500 €/Jahr	Banken, Behörden

> 💡 **MERKE – Self-Signed im Schullab**
> Im Schulnetzwerk und auf internen Servern ohne erreichbare Domain ist ein self-signed Zertifikat die praktikable Wahl. Der Browser zeigt eine Warnung (*„Ihre Verbindung ist nicht privat"*), weil er die CA nicht kennt – die Verschlüsselung selbst ist trotzdem aktiv. Die Warnung kann durch manuelles Hinzufügen des Zertifikats als vertrauenswürdig umgangen werden.

3.3 | Schlüsselpaar: Public Key und Private Key

TLS verwendet **asymmetrische Kryptografie**: Es gibt immer ein zusammengehöriges Schlüsselpaar.

Schlüssel	Datei	Zugriffsrecht	Funktion
Private Key	`server.key`	`600` – nur root	Entschlüsselt den Session-Key (geheim!)
Public Key (im Zertifikat)	`server.crt`	`644` – lesbar	Wird an Clients gesendet, verschlüsselt Session-Key

> ⚠️ **SICHERHEITSREGEL**
> Der **Private Key darf den Server niemals verlassen** und muss vor unbefugtem Zugriff geschützt werden. Ist der Private Key kompromittiert, muss sofort ein neues Zertifikat ausgestellt werden. Dateiberechtigung immer auf `600` setzen: `sudo chmod 600 /etc/ssl/private/server.key`

3.4 | HTTPS in Apache: das Modul `mod_ssl`

> 💡 **MERKE – mod_ssl**
> Apache kann TLS nicht „von Haus aus": Erst das Modul `mod_ssl` ermöglicht HTTPS-Verbindungen auf Port 443. Unter Ubuntu ist es im Paket `apache2` bereits enthalten und wird mit `sudo a2enmod ssl` aktiviert (danach `sudo systemctl restart apache2`). Im HTTPS-VirtualHost tragen die Direktiven `SSLCertificateFile` (Zertifikat) und `SSLCertificateKeyFile` (privater Schlüssel) die beiden Dateien ein.

4 | PKI – Public Key Infrastructure

Damit Browser einem Zertifikat vertrauen, muss es von einer bekannten **Certificate Authority (CA)** signiert sein. CA-Zertifikate sind in Browsern und Betriebssystemen vorinstalliert (Trust Store).

- **Root-CA:** Oberste Vertrauensinstanz (z. B. DigiCert, ISRG/Let's Encrypt) – im Browser vorinstalliert
- **Intermediate-CA:** Von der Root-CA autorisierte Zwischen-CA, die Serverzertifikate ausstellt
- **Server-Zertifikat:** Das eigentliche Zertifikat Ihres Webservers

Bei einem **self-signed Zertifikat** ist der Server gleichzeitig seine eigene CA – er ist im Trust Store der Browser nicht vorhanden, daher erscheint die Warnung.

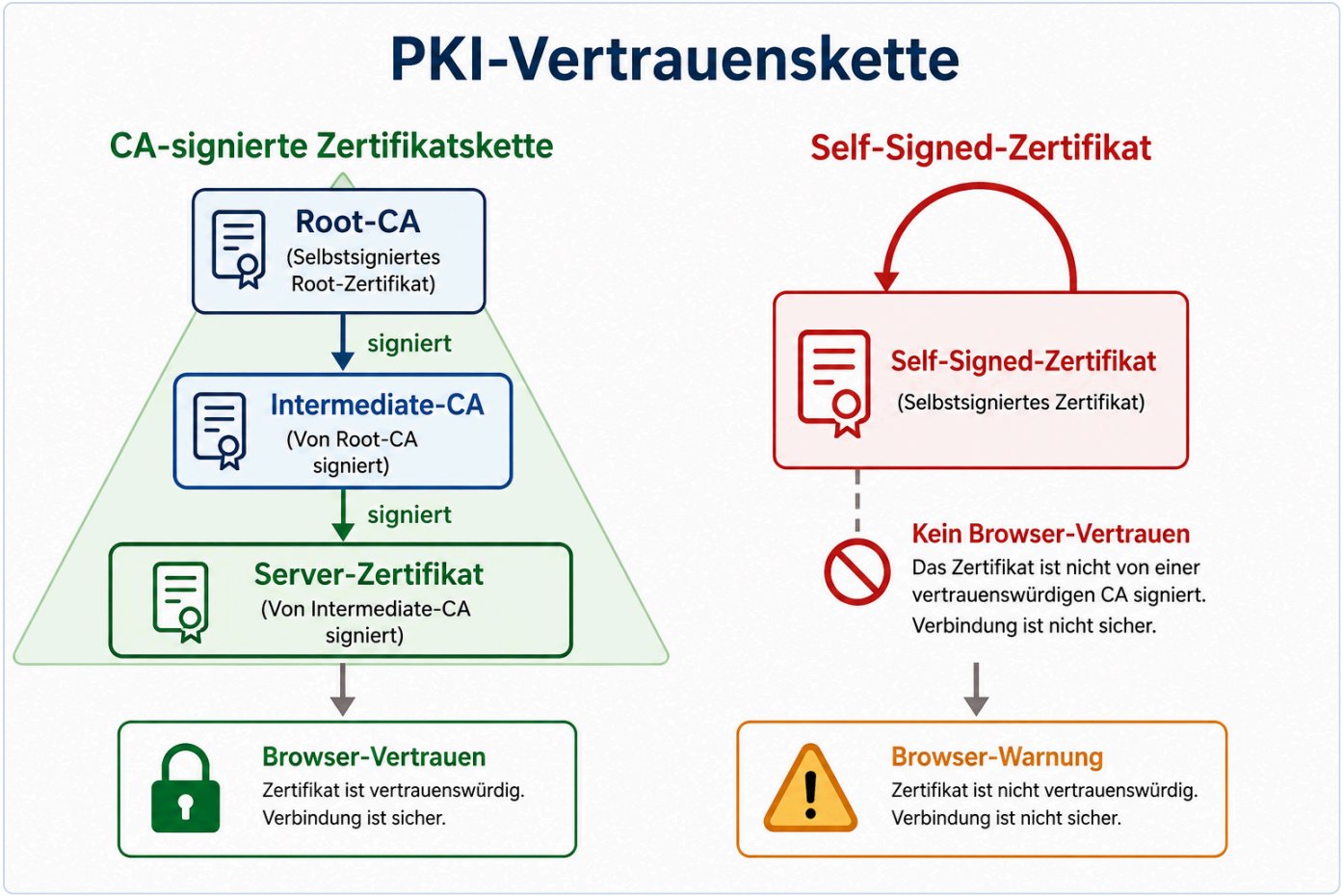


Abbildung 1: PKI-Vertrauenskette – CA-signiert vs. self-signed

5 | Zusammenfassung

ZUSAMMENFASSUNG – IB01

- HTTPS = HTTP + TLS-Verschlüsselung auf Port 443
- TLS schützt: Vertraulichkeit, Integrität, Authentizität
- Jeder Server benötigt ein Zertifikat mit Public Key und einen Private Key
- Self-Signed: kostenlos, kein Browser-Vertrauen, ideal für interne / Labor-Umgebungen
- CA-signiert (z. B. Let's Encrypt): kostenlos, vom Browser vertraut, für öffentliche Server
- Der Private Key ist streng vertraulich – Dateiberechtigung `600`
- Aktuell empfohlen: TLS 1.2 oder TLS 1.3